

Plan 004: Containerizar SQL Server para testes de integração

Executor instructions: Follow this plan step by step. Run every verification command and confirm the expected result before moving to the next step. If anything in the “STOP conditions” section occurs, stop and report — do not improvise. When done, update the status row for this plan in `plans/README.md`.

Drift check (run first): `git diff --stat 94369a2..HEAD -- tests/integration/ docker-compose.yml sql/` Se qualquer arquivo in-scope mudou desde `94369a2`, PARE.

Status

- **Priority:** P2
- **Effort:** M
- **Risk:** MED
- **Depends on:** none
- **Category:** tests
- **Planned at:** commit `94369a2`, 2026-06-28

Why this matters

Os testes de integração em `tests/integration/` existem mas são sempre ignorados — dependem de `PIFF_RUN_INTEGRATION=true` + SQL Server real com dados do Projeto_54. Sem um ambiente de banco reproduível, as queries SQL, os repositórios e as migrações de schema nunca são verificadas em teste automatizado.

Current state

- `tests/integration/test_sql_repositories.py` — 1 teste simples (`test_list_test_ids_integration_smoke`), sempre skipped:

```
pytestmark = pytest.mark.skipif(
    os.getenv("PIFF_RUN_INTEGRATION", "false").lower() != "true",
    reason="Teste de integracao depende de SQL Server local...",
)
```

- `tests/integration/test_db_queries.py` — testes parametrizados (6 cilindros × 4 funções = 24 cenários), sem skipif, mas falham sem banco. Único teste que realmente exercita os repositórios.
- `sql/` — contém scripts de criação do schema:
 - `03_create_projeto_54_ng.sql`
 - `04_create_projeto_54_hp.sql`, `04_create_projeto_54_hp_v2.sql`, `04_create_projeto_54_hp_v3.sql`
 - `05_create_projeto_54_hp_v2.sql`
- `docker-compose.yml` — não existe (precisa ser criado).
- Atualmente, conectar ao SQL Server requer configuração manual de variáveis de ambiente `PIFF_SQL_*`.

Commands you will need

Purpose	Command	Expected on success
Verificar Docker	<code>docker --version</code>	<code>Docker version XX.xx</code>
Verificar Compose	<code>docker compose version</code>	<code>Docker Compose version v2.xx</code>
Subir container	<code>docker compose up -d</code>	exit 0, container running
Testes integração	<code>PIFF_RUN_INTEGRATION=true python -m pytest tests/integration -q -s</code>	all pass
Testes unitários	<code>python -m pytest tests/unit -q</code>	12 passed

Scope

In scope:

- `docker-compose.yml` — criar na raiz
- `.env.example` — criar na raiz (documentar variáveis)
- `sql/seed/` — diretório para dados de seed (opcional)
- `tests/integration/conftest.py` — criar com fixture de conexão

Out of scope:

- `tests/integration/test_db_queries.py` — não modificar o conteúdo dos testes existentes, apenas o `conftest` e/ou markers
- `src/` — não tocar código de produção
- Dados de seed reais — criar dados sintéticos mínimos, não copiar o banco de produção

Git workflow

- Branch: `advisor/004-containerized-sql-server`
- Commits: um commit por step
- Mensagens: `feat: adiciona docker-compose para SQL Server de teste` / `test: adiciona conftest para testes de integração`

Steps

Step 1: Criar `docker-compose.yml`

Crie na raiz do projeto:

```
version: "3.8"

services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: piff54-sqlserver-test
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_SA_PASSWORD: "P@ssw0rd_Projeto54!"
      MSSQL_PID: Developer
    ports:
      - "1433:1433"
    volumes:
      - sqlserver-data:/var/lib/mssql/data
      - ./sql:/docker-entrypoint-initdb.d
    healthcheck:
      test: /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P "P@ssw0rd_Projeto54!" -C -Q "SELECT 1" || exit 1
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s

volumes:
  sqlserver-data:
```

Nota: O volume `./sql:/docker-entrypoint-initdb.d` mapeia os scripts de criação. O SQL Server não executa `docker-entrypoint-initdb.d` automaticamente como PostgreSQL — será necessário um passo extra (step 2) para executar os scripts.

Verifique: `docker compose config` → saída YAML válida

Step 2: Subir o container e criar o banco

```
docker compose up -d
# Aguardar healthcheck:
docker compose exec sqlserver bash -c "
  until /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P 'P@ssw0rd_Projeto54!' -C -Q 'SELECT 1' > /dev/null 2>&1; do
    echo 'Aguardando SQL Server...'
    sleep 3
  done
  echo 'SQL Server pronto'
"
```

Execute o script de criação do schema mais recente:

```
docker compose exec sqlserver bash -c "
  /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P 'P@ssw0rd_Projeto54!' -C -d master -i /docker-entrypoint-initdb.d/03_c
"
```

(Use `04_create_projeto_54_hp.sql` ou `03_create_projeto_54_ng.sql` dependendo de qual schema a aplicação usa. O `SETTINGS.sql_database` padrão é `Projeto_54`.)

Verifique:

```
docker compose exec sqlserver bash -c "
  /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P 'P@ssw0rd_Projeto54!' -C -Q 'SELECT name FROM sys.databases' | grep Pr
" → `Projeto_54`

### Step 3: Criar `.env.example`

```bash
cat > .env.example << 'EOF'
PIFF-0054-ng – Configuração via variáveis de ambiente
Copie para .env e ajuste os valores

SQL Server (padrões funcionam com docker-compose)
PIFF_SQL_SERVER=localhost
PIFF_SQL_DATABASE=Projeto_54
PIFF_SQL_USERNAME=sa
PIFF_SQL_PASSWORD=P@ssw0rd_Projeto54!
PIFF_SQL_DRIVER=ODBC Driver 17 for SQL Server
PIFF_SQL_TRUSTED_CONNECTION=false

Cache TTL (segundos)
PIFF_CACHE_TTL_SECONDS=300

Redis (opcional – app usa fallback em memória se indisponível)
PIFF_REDIS_HOST=localhost
PIFF_REDIS_PORT=6379
EOF
```

Verifique: `cat .env.example | head -3` → comentários + `PIFF_SQL_SERVER`

## Step 4: Ajustar testes de integração para usar conftest

Crie `tests/integration/conftest.py`:

```

"""Fixtures compartilhadas para testes de integração com SQL Server em container."""
from __future__ import annotations

import os
from pathlib import Path

import pytest

from src.config.settings import SETTINGS
from src.infrastructure.sql.connection import SqlServerConnection
from src.infrastructure.sql.repositories import (
 SqlDiagnosticsRepository,
 SqlObservationRepository,
 SqlTestQueryRepository,
)

@pytest.fixture(scope="session")
def conn():
 """Conexão compartilhada para a sessão de teste."""
 c = SqlServerConnection(SETTINGS)
 yield c

@pytest.fixture(scope="session")
def test_repo(conn):
 return SqlTestQueryRepository(conn)

@pytest.fixture(scope="session")
def obs_repo(conn):
 return SqlObservationRepository(conn)

@pytest.fixture(scope="session")
def diag_repo(conn):
 return SqlDiagnosticsRepository(conn)

```

Não modifique `test_sql_repositories.py` ou `test_db_queries.py` — deixe-os usando o fixture `repo` que já têm (em `test_db_queries.py`) ou a conexão direta (em `test_sql_repositories.py`). O `conftest` é apenas para referência futura. O `test_db_queries.py` já tem fixture `repo` próprio.

**Verifique:** `python -m pytest tests/integration/test_sql_repositories.py --setup-plan 2>&1 | head -5` → o teste ainda aparece como `skipped` (sem `PIFF_RUN_INTEGRATION=true`)

## Step 5: Rodar testes de integração

```
PIFF_RUN_INTEGRATION=true python -m pytest tests/integration -q -s --tb=short 2>&1 | tail -20
```

**Esperado:** todos os testes parametrizados passam (6 cilindros × 4 funções = 24 testes em `test_db_queries.py` + 1 em `test_sql_repositories.py`).

## Step 6: Atualizar `pytest.ini` com marcadores

```

[pytest]
testpaths = tests
pythonpath = .
markers =
 integration: testes que dependem de SQL Server em container

```

E adicione o marker nos testes de integração que ainda não têm:

```
tests/integration/test_db_queries.py – adicionar no módulo:
pytestmark = pytest.mark.integration
```

**Verifique:** `python -m pytest tests/integration -m integration --co 2>&1 | head -5` → lista os testes marcados como `integration`

## Test plan

---

- Nenhum teste novo além dos existentes em `tests/integration/`.
- O conftest criado é opcional (os testes existentes já funcionam com suas próprias fixtures).
- Testar manualmente: `PIFF_RUN_INTEGRATION=true python -m pytest tests/integration -q`

## Done criteria

---

- ☐ `[] docker compose up -d` sobe SQL Server 2022 em container
- ☐ `[]` Scripts SQL em `sql/` executam contra o container
- ☐ `[] .env.example` documenta as variáveis necessárias
- ☐ `[] PIFF_RUN_INTEGRATION=true python -m pytest tests/integration -q` → todos passam
- ☐ `[] python -m pytest tests/unit -q` → 12 passed (nada quebrado)

## STOP conditions

---

- Se o Docker não estiver disponível ( `docker --version` falha), PARE e reporte. Este plano requer Docker.
- Se o healthcheck do SQL Server não ficar verde após 5 minutos, PARE.
- Se os testes de integração falharem por dados ausentes (tabelas vazias), PARE — será necessário criar dados de seed sintéticos.
- Se `ODBC Driver 17 for SQL Server` não estiver instalado no host (para `pyodbc.connect` do Python), PARE e reporte. O driver precisa estar disponível para que os testes Python consigam se conectar.

## Maintenance notes

---

- O container SQL Server com imagem `mcr.microsoft.com/mssql/server:2022-latest` consome ~2GB de RAM em idle. Para ambientes com pouca memória, use `MSSQL_MEMORY_LIMIT_MB=1024`.
- Os dados no volume `sqlserver-data` persistem entre execuções do `docker compose down`. Para resetar: `docker compose down -v`.
- Se houver múltiplos schemas (ng, hp, hp\_v2, hp\_v3), considere criar bancos separados no container e scripts de seed para cada versão.
- Driver ODBC 18 também funciona — ajuste `PIFF_SQL_DRIVER` no `.env`.